Data Structures:

Data structures are a fundamental concept in computer science and programming that define the way data is organized and stored in memory. They provide a systematic way to manage, manipulate, and access data efficiently.

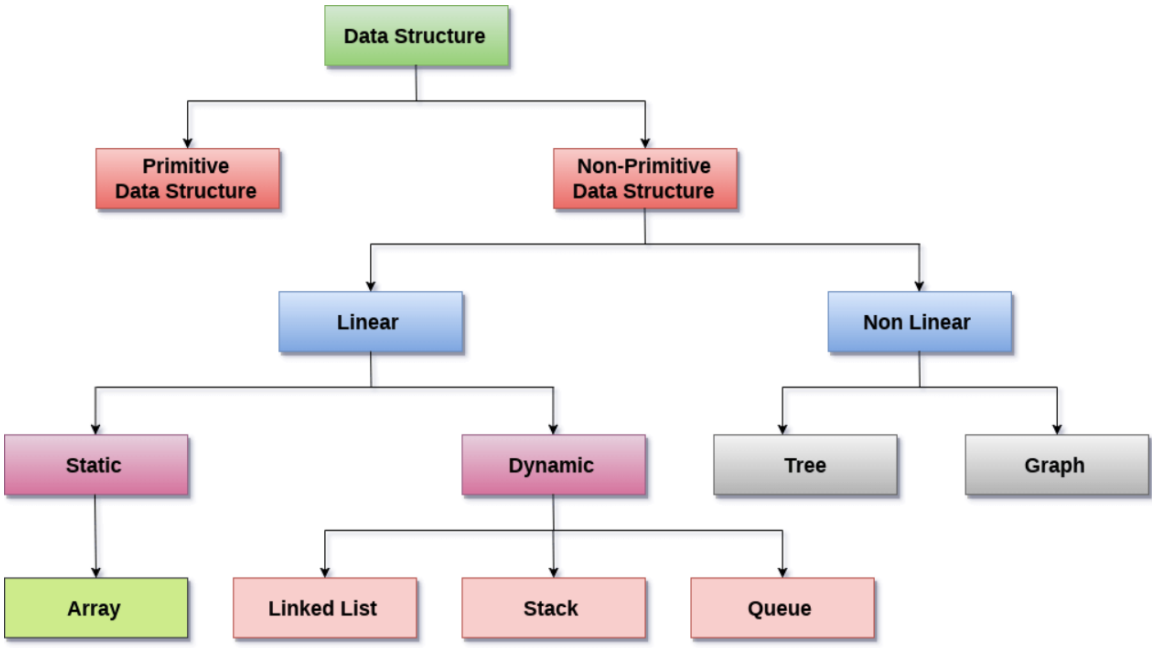
Data structures can vary in complexity and purpose, ranging from simple arrays and linked lists to more advanced structures like trees, graphs, and hash tables.

In essence, data structures consist of two main components:

1. Organization: Data structures specify how data elements are organized and arranged in memory. This organization determines how data can be accessed, inserted, deleted, and manipulated.
2. Operations: Data structures define a set of operations or functions that can be performed on the data, such as insertion, deletion, searching, sorting, and traversal. These operations are designed to provide efficient ways to interact with the data stored in the structure.

Classification of Data Structures:

The data structures can be classified as shown in the below diagram. Primarily, based on organization, the data structures are classified as Linear and Non-Linear Data Structures.



A linear data structure is a type of data structure where elements are arranged sequentially, one after the other.

In a linear data structure, each element has a unique predecessor and successor, except for the first and last elements. The order of elements is determined by their position in the sequence.

Examples of linear data structures include:

1. Arrays: A collection of similar type elements stored in contiguous memory locations, where each element is accessed by its index.
2. Linked Lists: A collection of nodes where each node contains a data element and a reference (or pointer) to the next node in the sequence.
3. Stacks: A Last-In-First-Out (LIFO) collection of elements where items are added and removed from the same end, typically referred to as the "top."
4. Queues: A First-In-First-Out (FIFO) collection of elements where items are added at the rear (enqueue) and removed from the front (dequeue).

Linear data structures are characterized by their simplicity, sequential arrangement, and predictable access patterns, making them essential components in various algorithms and applications.

Abstract Data types (ADT) and their implementation

Abstract Data Types (ADTs) are mathematical models for certain types of data structures, defining a set of operations and their behavior without specifying the implementation details.

ADTs focus on what operations can be performed on the data structure and what properties these operations must satisfy, rather than how they are implemented.

Here are some common abstract data types and their potential implementations:

Stack: Operations: push(), pop(), peek(), isEmpty()

- Implementation:
  - Using an array: Maintain a pointer to the top of the stack and resize the array as needed.
  - Using a linked list: Maintain a reference to the top node and update pointers accordingly.

Queue: Operations: enqueue(), dequeue(), peek(), isEmpty()

- Implementation:
  - Using an array: Maintain pointers to the front and rear of the queue and wrap around the array as elements are dequeued.
  - Using a linked list: Maintain references to the front and rear nodes and update pointers accordingly.

Linked List: Operations:

insert_begin(data): Inserts a new node containing the specified data at the beginning of the list.
insert_end(data): Inserts a new node containing the specified data at the end of the list.
insert_at_index(index, data): Inserts a new node containing the specified data at the given index in the list.
delete_begin(): Deletes the first node from the list.
delete_end(): Deletes the last node from the list.
delete_at_index(index): Deletes the node at the given index from the list.
size(): Returns the number of elements in the list.
clear(): Removes all elements from the list.

Tree:

- Operations: insert(), delete(), search(), traversal() (e.g., inorder(), preorder(), postorder())
- Implementation:
  - Binary Search Tree (BST): Maintain a binary tree structure where each node's left child contains values less than the node's value and the right child contains values greater than the node's value.
  - Balanced Tree (e.g., AVL tree, Red-Black tree): Ensure that the height of the tree remains balanced to ensure efficient operations.

Map (Dictionary):

- Operations: insert(), delete(), search() (lookup), isEmpty(), size()
- Implementation:
  - Hash table: Use a hash function to map keys to indices in an array, handling collisions with techniques like chaining or open addressing.
  - Binary search tree: Maintain a binary search tree where each node stores a key-value pair, allowing efficient search operations.

These implementations can vary in terms of efficiency, memory usage, and ease of implementation, depending on the specific requirements of the application and the trade-offs involved. The choice of implementation often depends on factors such as the expected size of the data, the frequency of operations, and the desired performance characteristics.

Nonlinear Data Structures:

A non-linear data structure is a type of data organization where data elements are not arranged sequentially or in a linear manner.

Unlike linear data structures, such as arrays or linked lists, where each element has a single predecessor and successor, non-linear data structures allow for more complex relationships between elements.

The key characteristic of non-linear data structures is that they can have multiple predecessors or successors, forming more intricate connections between data elements.

- **Trees**: Hierarchical structures consisting of nodes connected by edges with one root node and child nodes.
  - Binary Trees: Each node has at most two children.
  - Binary Search Trees (BSTs): Binary trees with the property that the left child of a node contains values less than the node's value, and the right child contains values greater than the node's value.
  - Balanced Trees (AVL, Red-Black Trees): Binary search trees with balanced heights to ensure efficient operations.
- **Graphs**: Networks consisting of nodes (vertices) and edges connecting these nodes.
  - Directed Graphs (Digraphs): Graphs with edges having a direction.
  - Undirected Graphs: Graphs with edges having no direction.
- **Heaps**: Complete binary trees that satisfy the heap property (e.g., min-heap, max-heap).
- **Hash Tables**: Data structures that store key-value pairs and use a hash function to map keys to indices, providing constant-time average-case access.

Importance of nonlinear data structures:

Non-linear data structures, such as trees and graphs, are essential in computer science and programming for several reasons:

1. **Hierarchical Representation:** Non-linear data structures allow the representation of hierarchical relationships between data elements. Trees, for example, can model hierarchical structures like file systems, organizational charts, or hierarchical data in databases.
2. **Efficient Searching:** Non-linear data structures often facilitate efficient searching operations. Binary search trees (BSTs), for instance, enable fast searching, insertion, and deletion operations with time complexity proportional to the height of the tree (O(log n) for balanced trees).
3. **Relationship Representation:** Graphs are versatile structures for representing relationships between objects. They are used in social networks, network routing algorithms, geographical maps, and dependency graphs in software systems.
4. **Optimization Problems:** Non-linear data structures are fundamental in solving optimization problems. Graph algorithms, such as Dijkstra's algorithm for finding the shortest path or Prim's and Kruskal's algorithms for finding minimum spanning trees, are crucial in various domains, including transportation, telecommunications, and logistics.
5. **Data Modelling:** Non-linear data structures provide powerful ways to model real-world scenarios where relationships between data elements are complex and interconnected. Trees and graphs allow for the representation of dependencies, constraints, and connections that may not be easily represented using linear structures.
6. **Memory Efficiency:** In some cases, non-linear data structures can offer memory efficiency. For example, in a sparse matrix representation, a graph can efficiently represent a matrix with a large number of zero elements.
7. **Algorithmic Solutions:** Many algorithmic problems and challenges are naturally represented and solved using non-linear data structures. Efficient algorithms for tasks such as pathfinding, network analysis, and optimization often rely on the expressive power and flexibility of trees and graphs.

Asymptotic analysis of an algorithm is a method used to analyze the performance and efficiency of algorithms as the input size grows towards infinity. It provides a way to describe the behavior of an algorithm in terms of its time complexity and space complexity without focusing on the specific details of hardware, programming language, or other implementation-specific factors.

There are two main aspects of asymptotic analysis: Time Complexity and Space Complexity

Time Complexity:

Time complexity is a measure of the amount of time an algorithm takes to complete as a function of the length of its input.

It describes the relationship between the input size and the number of basic operations performed by the algorithm.

Time complexity is typically expressed using big O notation, which provides an upper bound on the asymptotic growth rate of the running time.

Example notations:

- O(1): Constant time complexity. The running time of the algorithm remains constant regardless of the input size.
- O(log n): Logarithmic time complexity. The running time grows logarithmically as the input size increases.
- O(n): Linear time complexity. The running time grows linearly with the input size.
- O(n^2): Quadratic time complexity. The running time grows quadratically with the input size.
- O(2^n): Exponential time complexity. The running time grows exponentially with the input size.

Space Complexity:

Space complexity is a measure of the amount of memory space an algorithm requires as a function of the length of its input.

It describes the relationship between the input size and the amount of memory used by the algorithm.

Space complexity is typically expressed using big O notation, similar to time complexity.

Example notations:

- O(1): Constant space complexity. The amount of memory used remains constant regardless of the input size.
- O(n): Linear space complexity. The amount of memory used grows linearly with the input size.
- O(n^2): Quadratic space complexity. The amount of memory used grows quadratically with the input size.

These notations provide a concise way to describe the performance characteristics of algorithms and data structures, making it easier to analyze and compare their efficiency.

Big O Notation (O):

- Big O notation is used to describe the upper bound or worst-case scenario of the time or space complexity of an algorithm.
- It represents the maximum growth rate of an algorithm's running time or memory usage as the input size increases.

Omega Notation (Ω):

- Omega notation describes the lower bound or best-case scenario of the time complexity of an algorithm.
- It represents the minimum growth rate of an algorithm's running time as the input size increases.

Example:

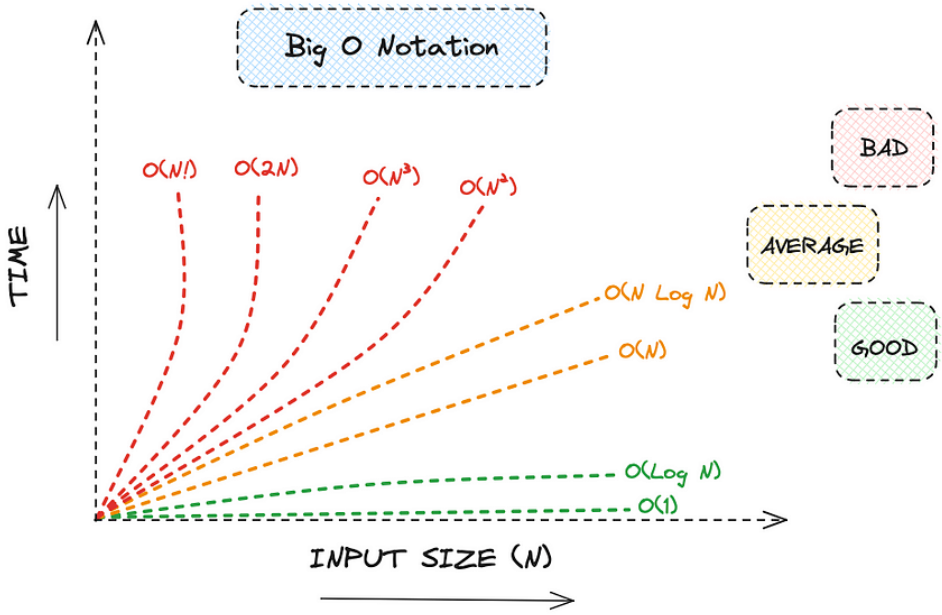
Ω(n): Linear time complexity. The algorithm's performance has a lower bound that grows linearly with the input size.

Theta Notation (θ):

- Theta notation describes both the upper and lower bounds of the time complexity of an algorithm, providing a tight bound on its performance.
- It represents the average-case scenario of an algorithm's running time as the input size increases.

Example:

θ(n): Linear time complexity. The algorithm's performance is bounded both from above and below and grows linearly with the input size.



Searching: It is the process of finding an element from the given List. In general there are two search algorithms, viz., Linear Search and Binary Search.

Linear Search: Definition and Explanation

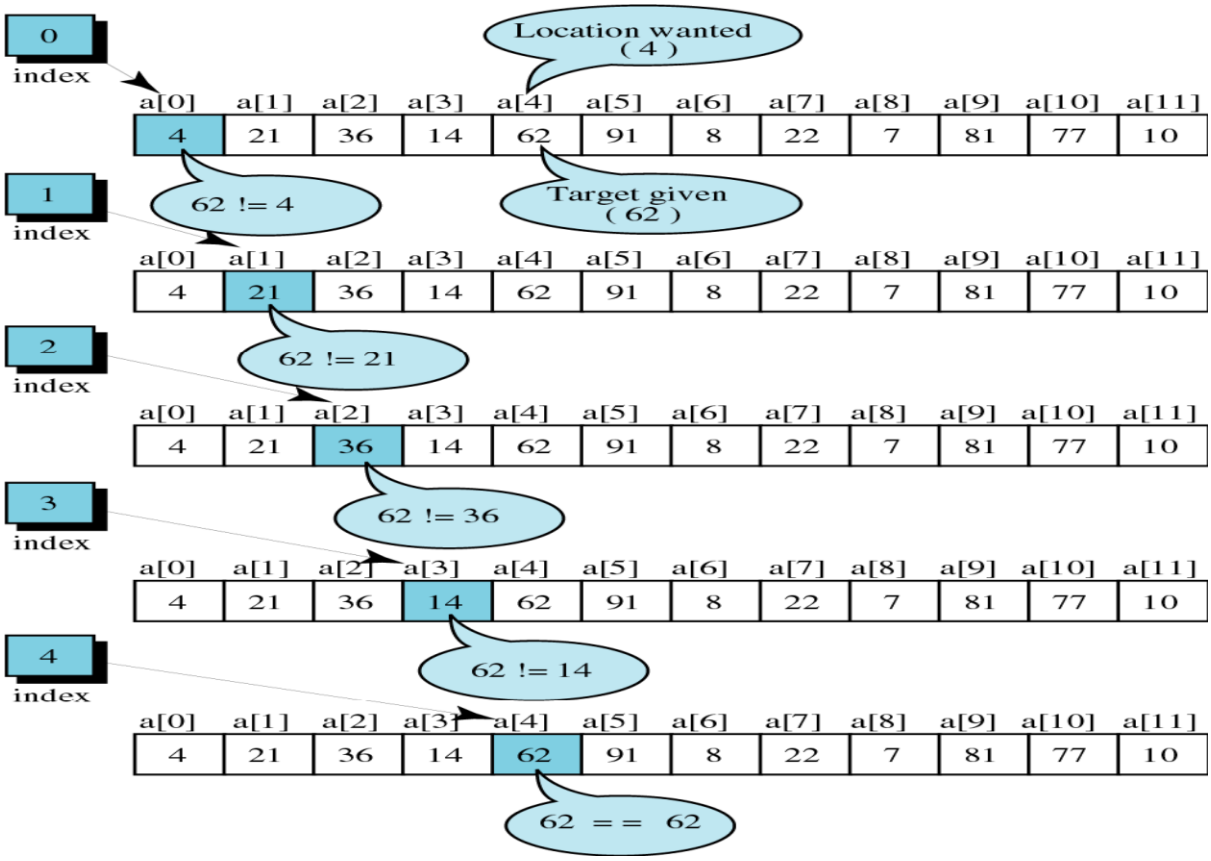
Linear search, also known as sequential search, is a simple search algorithm that checks every element in a list sequentially until the desired element is found or the list ends. It is used on a collection of items.

It operates by starting at the beginning of the list and comparing each element with the value being searched until a match is found or the end of the list is reached.

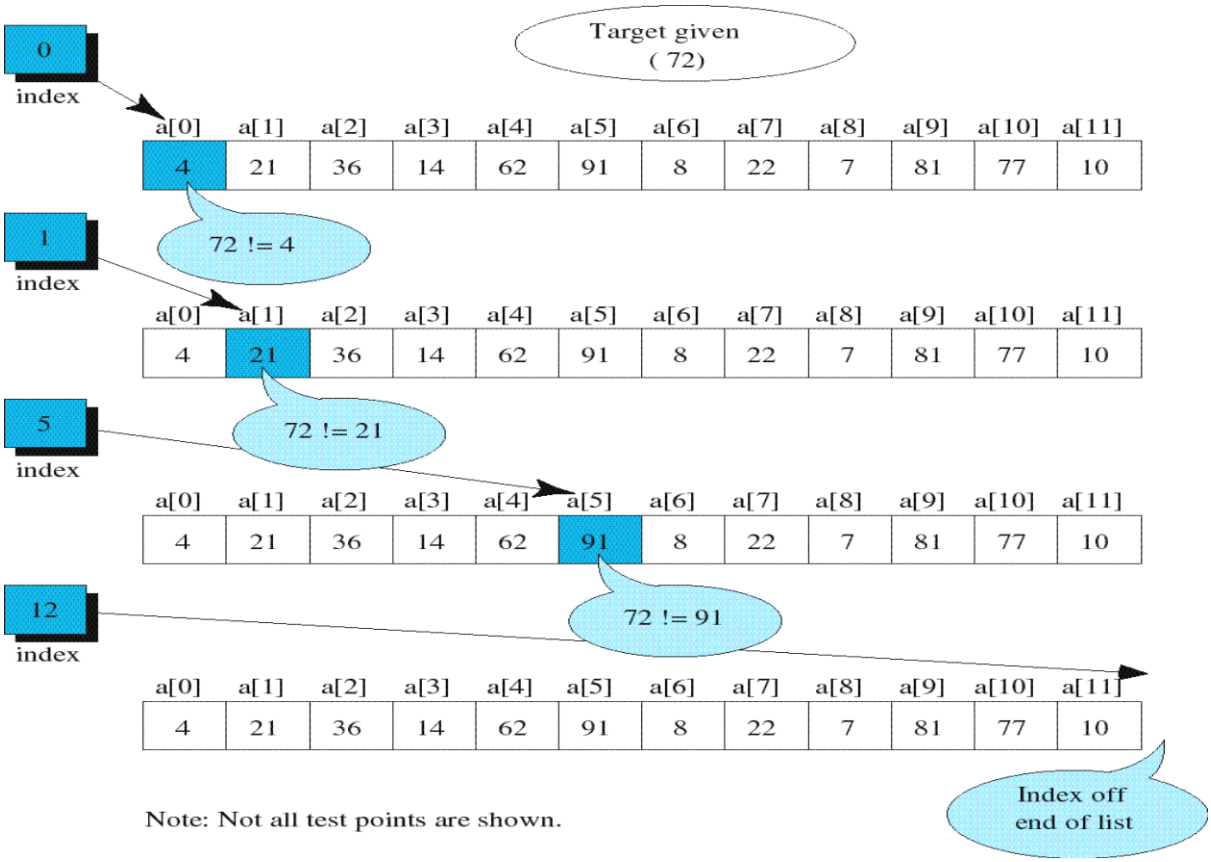
```
int linearSearch(int arr[], int size, int value) {
  for (int i = 0; i < size; i++) {
    if (arr[i] == value)
      return i; // Return the index of the found element
  }
  return -1; // Return -1 if the element is not found
```

In the provided C code, the **linearSearch** function takes an array **arr**, its size **size**, and the **value** to search for as arguments. It iterates through each element in the array using a for loop. If it finds the element that matches the **value**, it returns the index of that element. If it goes through the entire array without finding the **value**, it returns **-1** to indicate the element is not in the array.

Example: If we want to search for 62 from the given array, the successful search would be



And an unsuccessful search for 72 would be:

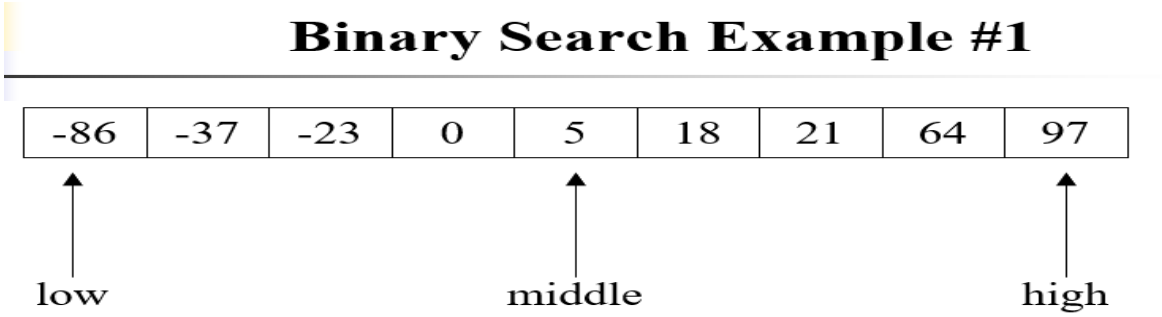


Note: Not all test points are shown.

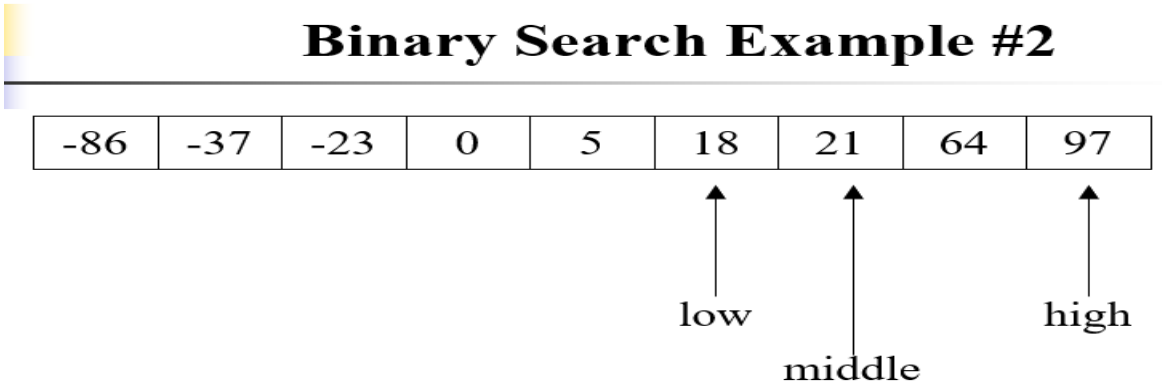
Binary Search:

Binary search is an efficient algorithm for finding an item from a sorted list of items. It works by repeatedly dividing in half the portion of the list that could contain the item, until the possible locations are reduced to just one.

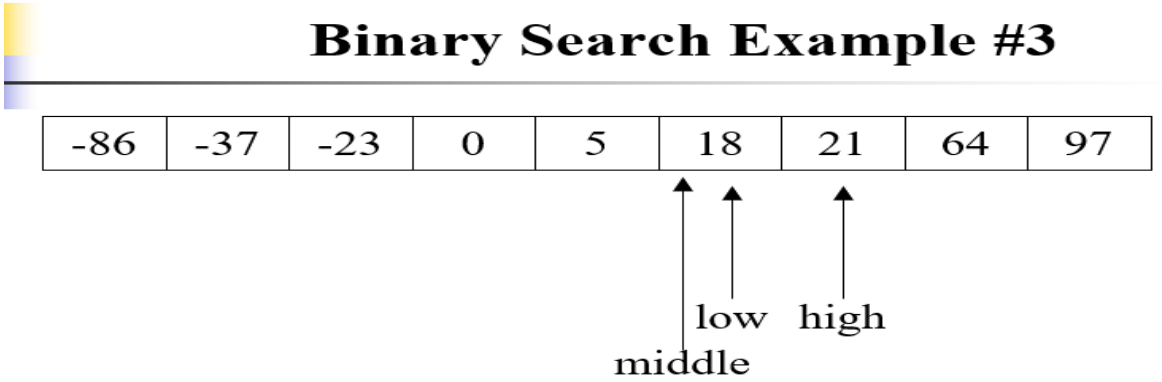
The key concept behind binary search is that it compares the target value to the middle element of the array; if they are not equal, the half in which the target cannot lie is eliminated, and the search continues on the remaining half until the target value is found or the search space is empty.



Searching for 18.



Searching for 18.



Searching for 18: found!

```
int binarySearch(int arr[], int l, int r, int x) {
  if (r >= l) {
    int mid = l + (r - l) / 2;

    // If the element is present at the middle itself
    if (arr[mid] == x)
      return mid;

    // If element is smaller than mid, then it can only be present in left subarray
    if (arr[mid] > x)
      return binarySearch(arr, l, mid - 1, x);

    // Else the element can only be present in right subarray
    return binarySearch(arr, mid + 1, r, x);
  }

  // We reach here when element is not present in array
  return -1;
}
```

In the binary search example above, the **binarySearch** function takes four arguments: the array **arr**, the starting position **l**, the ending position **r**, and the value **x** to be searched for.

1. The function calculates the middle index **mid** of the current search interval **[l..r]**.
2. It then compares the target value **x** with the value at the middle index.
    - If the target value is equal to the middle value, the position of **x** is found and its index is returned.
    - If the target value is less than the middle value, the function recursively searches the left sub-array **[l..mid-1]**.
    - If the target value is greater than the middle value, the function recursively searches the right sub-array **[mid+1..r]**.
3. This process continues until the value is found or the search interval is empty.

The time complexities for linear search and binary search:

1. **Linear Search:**
    - Time Complexity: O(n)
    - Description: In a linear search, the algorithm sequentially checks each element of the array until the desired element is found or the end of the array is reached. Therefore, in the worst-case scenario, the algorithm may need to examine every element in the array, resulting in a time complexity of O(n), where n is the number of elements in the array.
2. **Binary Search:**
    - Time Complexity: O(log n)
    - Description: In a binary search, the algorithm repeatedly divides the sorted array in half and compares the middle element with the target value. Based on the comparison, the algorithm eliminates half of the remaining elements at each step. Therefore, the time complexity of binary search is O(log n), where n is the number of elements in the array.

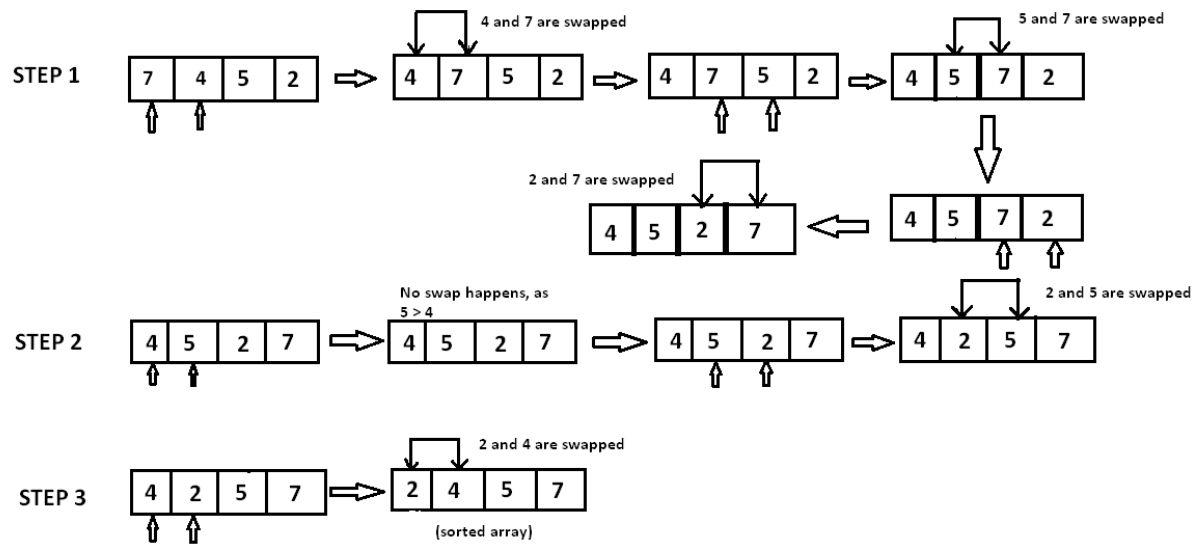
- Binary search is significantly faster than linear search for large datasets because it reduces the search space exponentially with each step. However, binary search requires the array to be sorted initially.

Sorting refers to the arrangement of data elements in ascending or descending order based on their values. Sorting is a fundamental operation in computer programming and is used extensively in various applications and algorithms.

Bubble Sort is a simple comparison-based sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted. It is named for the way smaller elements "bubble" to the top of the list with each iteration.

```
// Function to perform bubble sort
void bubbleSort(int arr[], int n) {
  int i, j, temp;
  for (i = 0; i < n-1; i++) {
    // Last i elements are already in place, so no need to check them
    for (j = 0; j < n-i-1; j++) {
      // Swap adjacent elements if they are in the wrong order
      if (arr[j] > arr[j+1]) {
        temp = arr[j];
        arr[j] = arr[j+1];
        arr[j+1] = temp;
      }
    }
  }
}
```

Let us take an example and see how it works. Assume that we have 4 elements in the array, they are 7 4 5 2. The following figure depicts the working principle of bubble sort.



1. **First Pass:**
    - Compare 7 and 4. Since 7 > 4, swap them: A[] = {4, 7, 5, 2}.
    - Compare 7 and 5. Since 7 > 5, swap them: A[] = {4, 5, 7, 2}.
    - Compare 7 and 2. Since 7 > 2, swap them: A[] = {4, 5, 2, 7}.
2. **Second Pass:**
    - Compare 4 and 5. No swap since 4 < 5.
    - Compare 5 and 2. Since 5 > 2, swap them: A[] = {4, 2, 5, 7}.
    - No need to compare the last pair since it's already in order.
3. **Third Pass:**
    - Compare 4 and 2. Since 4 > 2, swap them: A[] = {2, 4, 5, 7}.
    - The rest of the array is already sorted, so no further swaps are needed.

At this point, the array is sorted in ascending order: A[] = {2, 4, 5, 7}. This example demonstrates how bubble sort iteratively moves the largest unsorted element to its correct position by comparing and swapping adjacent elements.

Selection Sort is a simple comparison-based sorting algorithm that divides the input list into two sublists: a sorted sublist and an unsorted sublist. The algorithm repeatedly finds the minimum (or maximum) element from the unsorted sublist and swaps it with the first element of the unsorted sublist. This process continues until the entire list is sorted.

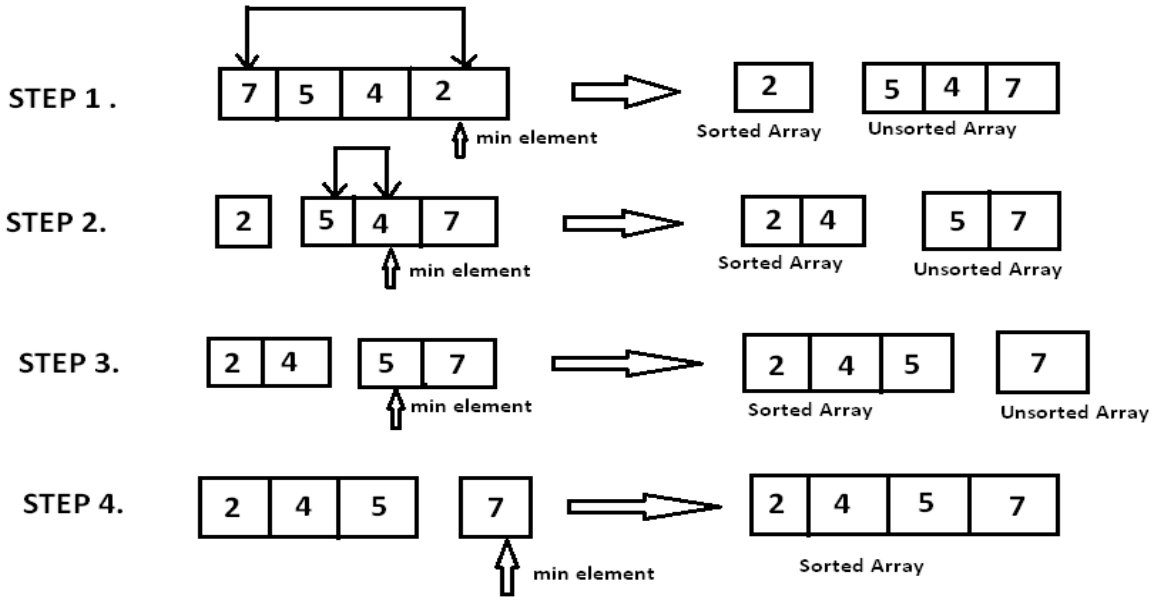
```
// Function to perform Selection Sort
void selectionSort(int arr[], int n) {
  int i, j, min_index, temp;
  for (i = 0; i < n-1; i++) {
    // Find the index of the minimum element in the unsorted part of the array
    min_index = i;
    for (j = i+1; j < n; j++) {
      if (arr[j] < arr[min_index]) {
        min_index = j;
      }
    }
    // Swap the minimum element with the first element of the unsorted part
    temp = arr[i];
    arr[i] = arr[min_index];
    arr[min_index] = temp;
  }
}
```

- The **selectionSort** function sorts the input array **arr** in ascending order using the Selection Sort algorithm.
- The outer loop iterates through each element of the array, representing the sorted sublist.

- Within each iteration of the outer loop, the inner loop finds the index of the minimum element in the unsorted part of the array.
- The minimum element is then swapped with the first element of the unsorted part.
- The process continues until the entire array is sorted.

Let's take the example array {7, 5, 4, 2} and go through the process of Selection Sort step by step:

1. **Initial Array:** {7, 5, 4, 2}
2. **First Pass (i = 0):**
    - The minimum element in the unsorted part of the array is 2, located at index 3.
    - Swap 7 (at index 0) with 2.
    - Updated Array: {2, 5, 4, 7}
3. **Second Pass (i = 1):**
    - The minimum element in the unsorted part of the array is 4, located at index 2.
    - Swap 5 (at index 1) with 4.
    - Updated Array: {2, 4, 5, 7}
4. **Third Pass (i = 2):**
    - The minimum element in the unsorted part of the array is 5, located at index 2 (already sorted).
    - No swap is needed.
    - Array remains unchanged: {2, 4, 5, 7}
5. **Fourth Pass (i = 3):**
    - The entire array is now sorted, and no further passes are needed.
    - Array remains unchanged: {2, 4, 5, 7}
6. **Sorted Array:** {2, 4, 5, 7}



Insertion Sort is a simple comparison-based sorting algorithm that builds the final sorted array (or list) one element at a time. It is particularly efficient for sorting small datasets or nearly sorted datasets.

Here's how the Insertion Sort algorithm works:

1. **Start with the second element:** Assume the first element in the array is already sorted. Start iterating from the second element.
2. **Compare and Insert:** For each element in the unsorted part of the array, compare it with the elements to its left in the sorted part. Move elements greater than the current element one position to the right to make space for inserting the current element in its correct position in the sorted part.
3. **Repeat:** Continue this process until all elements are sorted.

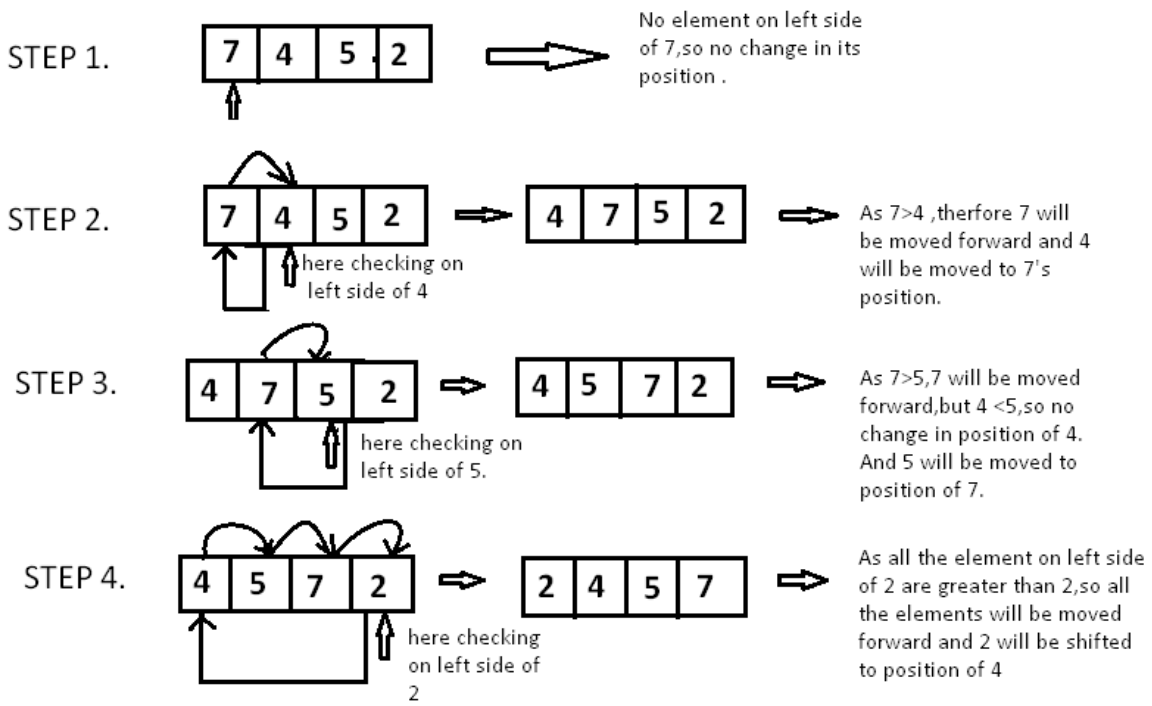
The key idea behind Insertion Sort is to iteratively insert each element into its correct position in the sorted part of the array, thereby gradually building the sorted array. At each iteration, the sorted part of the array increases by one element.

```
void insertionSort(int arr[], int n) {
  int i, j, key;
  for (i = 1; i < n; i++) {
    key = arr[i];
    j = i - 1;
    // Move elements of arr[0..i-1], that are greater than key, to one position ahead of their current
position
    while (j >= 0 && arr[j] > key) {
      arr[j + 1] = arr[j];
      j = j - 1;
    }
    arr[j + 1] = key;
  }
}
```

- The **insertionSort** function sorts the input array **arr** in ascending order using the Insertion Sort algorithm.
- The outer loop iterates through each element of the array starting from the second element (index 1).
- Within each iteration of the outer loop, the current element is compared with the elements to its left.
- If an element to the left of the current element is greater than the current element, it is moved one position to the right.
- This process continues until the correct position for the current element is found, and the current element is inserted into its correct position.
- The process repeats until the entire array is sorted.

Let's use the example array {7, 5, 4, 2} to illustrate how the Insertion Sort algorithm works step by step:

1. **Initial Array:** {7, 5, 4, 2}
2. **First Pass (i = 1):**
    - Current Element: 5
    - Compare 5 with 7 (the element to its left).

- Since 5 < 7, move 7 one position to the right.
- Updated Array: {5, 7, 4, 2}

3. **Second Pass (i = 2):**
   - Current Element: 4
   - Compare 4 with 7 and move 7 one position to the right.
   - Compare 4 with 5 and move 5 one position to the right.
   - Insert 4 in the correct position.
   - Updated Array: {4, 5, 7, 2}
4. **Third Pass (i = 3):**
   - Current Element: 2
   - Compare 2 with 7 and move 7 one position to the right.
   - Compare 2 with 5 and move 5 one position to the right.
   - Compare 2 with 4 and move 4 one position to the right.
   - Insert 2 in the correct position.
   - Updated Array: {2, 4, 5, 7}
5. **Array is Sorted:**
   - All elements are in their correct positions.
   - Array is sorted in ascending order: {2, 4, 5, 7}